Cryptography

Salsa20 and ChaCha20

Ksawery Możdżyński

Agenda For Today

1. **A keystream from a counter**
   Jump to any block
2. **Three simple operations**
   Add, rotate, XOR
3. **Salsa20**
   A 4 by 4 table of words
4. **ChaCha20: one block**
   Twenty rounds, step by step
5. **Why twenty rounds?**
   Measure it, then break it
6. **Fast on any processor**
   No special hardware needed
7. **ChaCha20-Poly1305**
   Where you use it every day

Today you will need



Python 3

nothing to install,
no extra libraries



An editor

any one you like



Paper and a pen

for predictions before you run the code

The tasks: **GitHub** Lecture2 / Task1 . . . Task9

Recall: writing bytes in hexadecimal

One byte = 8 bits = two hexadecimal digits (0 to 9, then A to F).



hex: written `0x48` or `48`

decimal: 72, the letter `H`

A 32-bit number is 4 bytes, so 8 hex digits: `0x61707865`

Recall: XOR

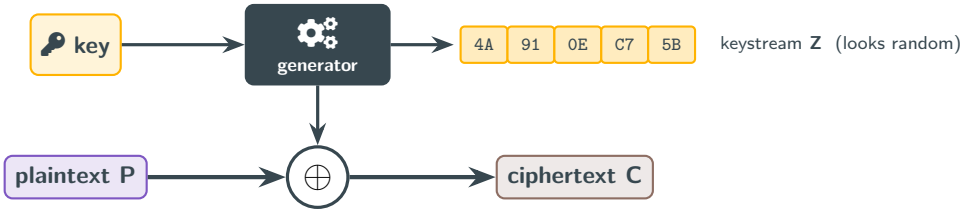
a	b	a XOR b
0	0	0
0	1	**1**
1	0	**1**
1	1	0

Same bits give 0, different bits give 1.
In Python: `a ^ b`

Twice with the same value changes nothing
$(a \text{ XOR } k) \text{ XOR } k = a$

XOR with the same value twice gives back what we started with.

Recall: a stream cipher

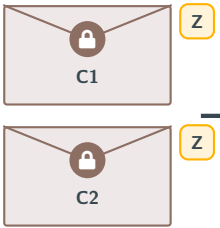


$C = P \text{ XOR } Z$ $\qquad$ $P = C \text{ XOR } Z$

Encryption and decryption are the same XOR with the same keystream.

Recall: never the same keystream twice

Two messages encrypted with the same keystream Z:



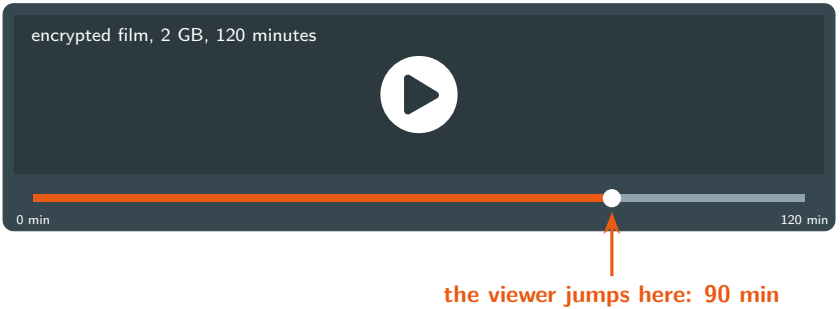
C1 XOR C2 = (P1 XOR Z) XOR (P2 XOR Z) = **P1 XOR P2**

The keystream disappears. Knowing P1 gives P2.

So every message gets a public value, the **nonce**, that changes the keystream.

One key, many messages: each message needs its own nonce.

A film on a server



To decrypt minute 90, we need the keystream for minute 90.

A generator that only goes forward

A generator like **RC4** gives the keystream one byte after another.
To reach minute 90 it must first produce and throw away 1.5 GB of keystream.



measured RC4 speed on a laptop:	0.57 GB/s
time before the film can play:	about **2.6 s**, for every jump

What do we really need?

Any piece of the keystream,
without computing the pieces before it.



We want to jump straight to any part of the keystream.

The idea: number the blocks

Cut the keystream into blocks of **64 bytes** and give each block a number.



Block n is computed from three things only: key, nonce, n

Block n does not depend on blocks 0 to $n - 1$.

The counter

Counter: the number of a 64-byte block of the keystream. It goes 0, 1, 2, ... for the blocks of one message. Like the nonce, it is public.

message:	bytes 0 to 63	bytes 64 to 127	bytes 128 to 191	...
counter:	0	1	2	...

Byte number `b` of the message is in block `b // 64`

Salsa20 and ChaCha20 from the outside



Two ciphers built on this idea, by Daniel J. Bernstein:
Salsa20 (2005) and **ChaCha20** (2008), an improved Salsa20.

Same key and nonce, a new counter: a new block.

Task 1: use ChaCha20

Task 1
You received an encrypted message and you know the key and the nonce.

key	`c0ffee00 c0ffee00 ...` (32 bytes)
nonce	`0000004a 00000000 00000007`
ciphertext	`36323fcd0cbf7202049805976e0d4afda0a2bc363f02eea05b...`

The message was encrypted starting with **counter 1**.

What does it say?

Task 1: what to do

```python
from chacha20_ready import chacha20

key = bytes.fromhex("c0ffee00" * 8)              # 32 bytes
nonce = bytes.fromhex("0000004a0000000000000007")   # 12 bytes
ciphertext = bytes.fromhex(
    "36323fcd0cbf7202049805976e0d4afda0a2bc36"
    "3f02eea05b49125427ad01f9709e3275ef98d8"
)

# TODO: decrypt the ciphertext with
#       chacha20(key, nonce, counter, data).
#       The message was encrypted starting with counter 1.
#       Print the plaintext as text: plaintext.decode()
```

Task 1

Open `use_chacha.py` from **GitHub**.

1. Call `chacha20` with counter 1.
2. Print the result as text.

Solution: use ChaCha20

```
plaintext = chacha20(key, nonce, 1, ciphertext)
print(plaintext.decode())
```

The output

```
The meeting moves to room 204 at 15:30.
```

To decrypt, call the same function again.

Task 2: jump to the middle

Task 2

`film.bin`: an encrypted film of 1000 blocks, 64 bytes each. It was encrypted starting with counter 0: block n used counter n.



The program already decrypts the whole film and prints block 700.

Decrypt only block 700, without blocks 0 to 699, and compare the times.

Task 2: what to do

```python
film = open("film.bin", "rb").read()       # 1000 blocks of 64 bytes
# Encrypted starting with counter 0: block n used counter n.

start = time.perf_counter()
everything = chacha20(key, nonce, 0, film)
print("whole film:", everything[700 * 64:701 * 64].decode().strip())
print(f"  took {1000 * (time.perf_counter() - start):.1f} ms")

start = time.perf_counter()
# TODO: decrypt ONLY block 700
#       (bytes 700*64 to 701*64 of the film).
#       Which counter do you need?
one_block = b""
```

Shortened on the slide: the imports, the key, the nonce and the last two prints are in the file.

Task 2
Open `jump.py` from **GitHub**.

1. Cut block 700 out of the film.
2. Decrypt it with the right counter.

In Python
a slice of bytes: `film[start:end]`

Solution: jump to the middle

```
one_block = chacha20(key, nonce, 700, film[700 * 64:701 * 64])
```



both print: `Frame 0700:  the hero finally opens the red door.`

The counter lets us decrypt any block on its own.

Nonce and counter: two different jobs

	changes between ...	who chooses it
nonce	messages sent with one key	the sender, once per message
counter	blocks of one message	counts 0, 1, 2, ... by itself

Both are public. The **key** is the only secret.

A (nonce, counter) pair must never repeat with the same key.

How long can one message be?

In ChaCha20 as used on the internet (**RFC 8439**):

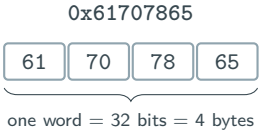


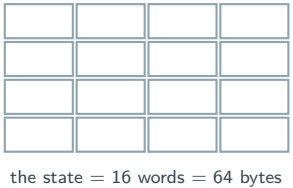
for one key and nonce

After 256 GiB, a new nonce is needed.

Inside: only 32-bit words

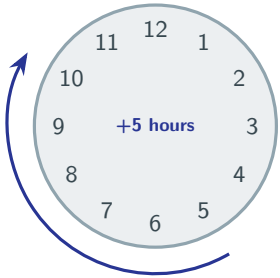
ChaCha20 does not work on single bytes. It works on **words**.





Everything inside happens on these 16 words.

Recall: mod



a **mod** *n*
the remainder after dividing *a* by *n*.
`7 mod 4 = 3`, `8 mod 4 = 0`

A clock
10 o'clock + 5 hours = 3 o'clock
`(10 + 5) mod 12 = 3`

In Python
`7 % 4`

Operation 1: addition mod 2^{32}

A word has 32 bits. A carry out of bit 31 is **lost**.

	0xFFFFFFFF	1111 1111 ... 1111
+	0x00000001	0000 0000 ... 0001
=	0x00000000	~~1~~ 0000 0000 ... 0000

the 33rd bit is dropped

It is addition mod 2^{32}: the clock has 4 294 967 296 positions.

Addition in Python

Python integers have no limit, so we cut the result to 32 bits ourselves:

```
(a + b) & 0xFFFFFFFF
```

`0xFFFFFFFF` is 32 ones. `&` keeps only the lowest 32 bits.

```
0xFFFFFFFF + 1                  = 0x100000000
(0xFFFFFFFF + 1) & 0xFFFFFFFF   = 0x00000000
```

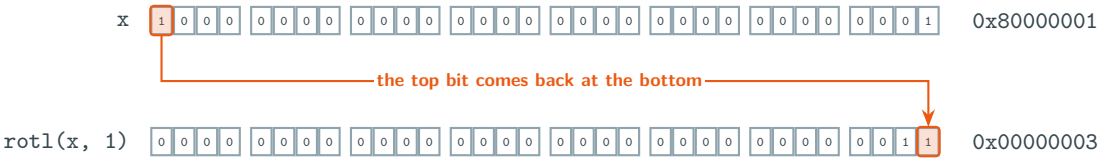
Operation 2: XOR on words

The same XOR as for bytes, on 32 bits at once:

	0x12131415	0001 0010 0001 0011 0001 0100 0001 0101
XOR	0x01234567	0000 0001 0010 0011 0100 0101 0110 0111
=	0x13305172	0001 0011 0011 0000 0101 0001 0111 0010

Operation 3: rotation

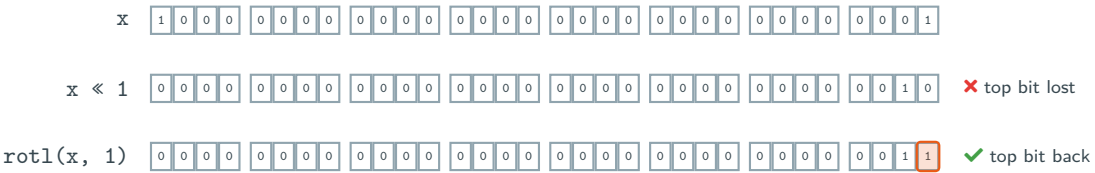
Rotate left by n: the bits that fall out on the left come back on the right.



No bit is lost, the word just turns around.

Rotation moves bits to new positions without losing any.

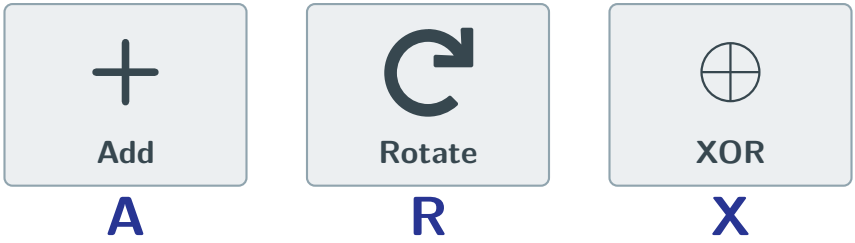
Rotation is not a shift



In Python, a rotation by n is built from two shifts:

```
((x « n) | (x » (32 - n))) & 0xFFFFFFFF
```

ARX



Three operations that every processor does in one step.
No tables in memory, no special instructions.

Salsa20 and ChaCha20 are only these three operations, repeated many times.

Task 3: write the operations

Task 3
Write two functions on 32-bit words. The program checks six examples and prints OK or FAIL for each.

`add32(a, b)`
addition mod 2^{32}
`add32(0xFFFFFFFF, 0x00000001)` $\rightarrow$ `0x00000000`

`rotl32(x, n)`
rotation left by n bits
`rotl32(0x80000001, 1)` $\rightarrow$ `0x00000003`

Task 3: what to do

```
MASK = 0xFFFFFFFF   # 32 ones: keeps only the lowest 32 bits


def add32(a, b):
    # TODO: add a and b and keep only 32 bits
    pass


def rotl32(x, n):
    # TODO: rotate the 32-bit word x left by n bits
    pass
```

Shortened on the slide: the six tests and longer comments are in the file.

Task 3

Open `arx.py` from **GitHub**.

One line of the output

```
OK  rotl32:  got 0x00000003,
    expected 0x00000003
```

At the start all six tests give `FAIL`.

Solution: write the operations

```python
def add32(a, b):
    return (a + b) & MASK

def rotl32(x, n):
    return ((x << n) | (x >> (32 - n))) & MASK
```

The output

OK six times

Why mix all three?

XOR and rotation alone

each output bit = XOR of some input bits

```
y5 = x2 ^ x9 ^ x30
```

the attacker can solve it like a set of equations

Addition

the carry depends on two bits at once

```
carry = x & y
```

(1 only if both are 1)

this is **not** a XOR of bits any more

Addition breaks the equations. XOR and rotation spread the bits.

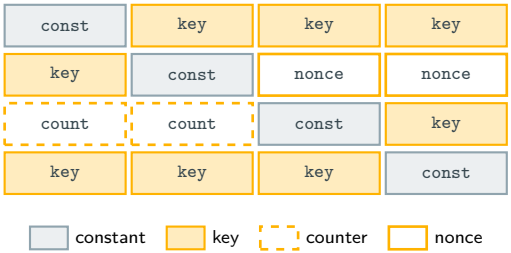
The state: a 4 by 4 table

The 16 words of the state are drawn as a 4 by 4 table:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

The numbers are the **positions** of the words, 0 to 15.

Salsa20: what goes where



key	8 words = 256 bits
nonce	2 words = 64 bits
counter	2 words = 64 bits
const	4 words on the diagonal

The constants

The four constant words are ASCII text:

0x61707865	0x3320646e	0x79622d32	0x6b206574
"expa"	"nd 3"	"2-by"	"te k"

"expand 32-byte k"

A fixed part of the state that nobody can choose.

The Salsa20 quarter round

quarter round of four words a, b, c, d

```
b ^= rotl(a + d, 7)
c ^= rotl(b + a, 9)
d ^= rotl(c + b, 13)
a ^= rotl(d + c, 18)
```

Every line
add two words, rotate, XOR into the third.

+
is addition mod 2^{32}.

A **quarter round** mixes 4 of the 16 words.

The quarter round mixes four words with ARX.

Task 4: one quarter round of Salsa20

Task 4

Input: `a = 0x00000001, b = 0, c = 0, d = 0`. Write the four lines and print the words after each line.



Predict first: the input has a single 1 bit.
How many 1 bits will the four words have at the end?

Task 4: what to do

```python
def salsa_quarter_round(a, b, c, d):
    # TODO: the four lines of the Salsa20 quarter round
    #   b ^= rotl(a + d, 7)
    #   c ^= rotl(b + a, 9)
    #   d ^= rotl(c + b, 13)
    #   a ^= rotl(d + c, 18)
    # use add32 for + and rotl32 for rotl;
    # print a, b, c, d after every line
    return a, b, c, d
```

Task 4

Open `salsa_qr.py` from **GitHub**.

Ready in the file

`add32` and `rotl32`

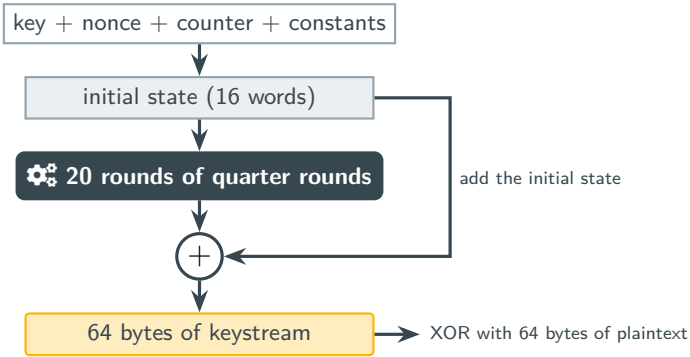
Solution: one quarter round of Salsa20

	a	b	c	d
start	0x00000001	0x00000000	0x00000000	0x00000000
line 1	0x00000001	0x00000080	0x00000000	0x00000000
line 2	0x00000001	0x00000080	0x00010200	0x00000000
line 3	0x00000001	0x00000080	0x00010200	0x20500000
line 4	0x08008145	0x00000080	0x00010200	0x20500000

1 bit in, **12 bits** out, after one quarter round.

One bit already reaches all four words.

From the state to a block



Salsa20 today



eSTREAM: a European competition for stream ciphers. In 2008 it recommends Salsa20/12 (12 rounds) for software.

Used in the NaCl and libsodium libraries (as XSalsa20), for example in Keybase chat.

ChaCha20 keeps the idea and changes the details.

What ChaCha20 changes

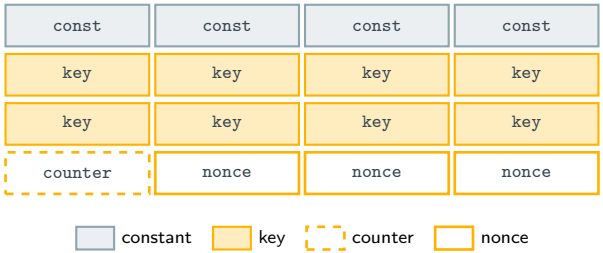
1. **A new quarter round**
each word changes twice, rotations 16, 12, 8, 7.

2. **A new layout**
the constants in the first row, the key in rows 2 and 3.

3. **Rounds alternate**
columns, then diagonals.

More mixing per round, the same three operations.

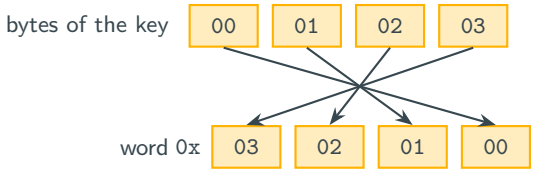
The ChaCha20 state



constants 128 bits key 256 bits counter 32 bits nonce 96 bits

Little-endian

Four bytes become one word with the **lowest byte first**:



In Python: `struct.unpack("<I", b"\x00\x01\x02\x03")` → `0x03020100`

This is the order used by most processors.

The state of the RFC example

key = 00 01 02 ... 1f, counter = 1, nonce = 00 00 00 09 00 00 00 4a 00 00 00 00

61707865	3320646e	79622d32	6b206574
03020100	07060504	0b0a0908	0f0e0d0c
13121110	17161514	1b1a1918	1f1e1d1c
00000001	09000000	4a000000	00000000

The ChaCha20 quarter round

quarter round of four words a, b, c, d

```
a += b;  d ^= a;  d = rotl(d, 16)
c += d;  b ^= c;  b = rotl(b, 12)
a += b;  d ^= a;  d = rotl(d, 8)
c += d;  b ^= c;  b = rotl(b, 7)
```

Every line
add, XOR, rotate. += is addition mod 2^{32}.

Each of the four words changes twice
In Salsa20 every word changed once.

Quarter round: the start

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

RFC 8439, section 2.1.1:
the test vector of the quarter round

a	b	c	d
0x11111111	0x01020304	0x9b8d6f43	0x01234567

Quarter round, line 1

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

```
a = 0x11111111 + 0x01020304 = 0x12131415
d = 0x01234567 ^ 0x12131415 = 0x13305172
d = rotl(0x13305172, 16)  = 0x51721330
```

a	b	c	d
0x12131415	0x01020304	0x9b8d6f43	0x51721330

Quarter round, line 2

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

```
c = 0x9b8d6f43 + 0x51721330 = 0xecff8273
b = 0x01020304 ^ 0xecff8273 = 0xedfd8177
b = rotl(0xedfd8177, 12)    = 0xd8177edf
```

a	b	c	d
0x12131415	0xd8177edf	0xecff8273	0x51721330

Quarter round, line 3

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

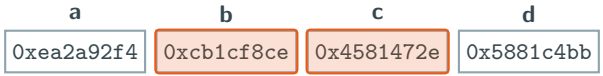
```
a = 0x12131415 + 0xd8177edf = 0xea2a92f4
d = 0x51721330 ^ 0xea2a92f4 = 0xbb5881c4
d = rotl(0xbb5881c4, 8)     = 0x5881c4bb
```

a	b	c	d
0xea2a92f4	0xd8177edf	0xecff8273	0x5881c4bb

Quarter round, line 4

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

```
c = 0xecff8273 + 0x5881c4bb = 0x4581472e
```
(a carry was lost)
```
b = 0xd8177edf ^ 0x4581472e = 0x9d9639f1
b = rotl(0x9d9639f1, 7)    = 0xcb1cf8ce
```



The result matches RFC 8439, section 2.1.1.

Task 5: write the quarter round

Task 5
Write the ChaCha20 quarter round on four words. Test it on the example from RFC 8439:

in	`0x11111111`	`0x01020304`	`0x9b8d6f43`	`0x01234567`
out	`0xea2a92f4`	`0xcb1cf8ce`	`0x4581472e`	`0x5881c4bb`

Task 5: what to do

```python
def quarter_round(a, b, c, d):
    # TODO: the four lines of the ChaCha20 quarter round
    #   a += b; d ^= a; d = rotl(d, 16)
    #   c += d; b ^= c; b = rotl(b, 12)
    #   a += b; d ^= a; d = rotl(d, 8)
    #   c += d; b ^= c; b = rotl(b, 7)
    # use add32 for += and rotl32 for rotl
    return a, b, c, d
```

Task 5

Open `quarter_round.py` from **GitHub**.

Ready in the file

`add32` and `rotl32`. The program prints OK or FAIL.

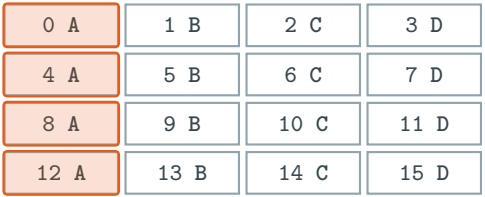
Solution: write the quarter round

```python
def quarter_round(a, b, c, d):
    a = add32(a, b); d ^= a; d = rotl32(d, 16)
    c = add32(c, d); b ^= c; b = rotl32(b, 12)
    a = add32(a, b); d ^= a; d = rotl32(d, 8)
    c = add32(c, d); b ^= c; b = rotl32(b, 7)
    return a, b, c, d
```

The output

```
got:      0xea2a92f4 0xcb1cf8ce
          0x4581472e 0x5881c4bb
OK
```

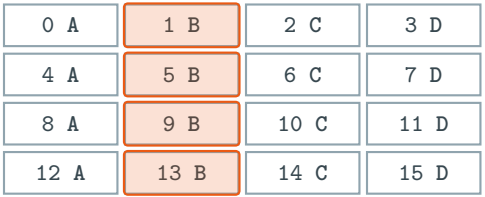
A column round




```
A: quarter_round(0, 4, 8, 12)
B: quarter_round(1, 5, 9, 13)
C: quarter_round(2, 6, 10, 14)
D: quarter_round(3, 7, 11, 15)
```

Column round: four quarter rounds, each on one column of the state.

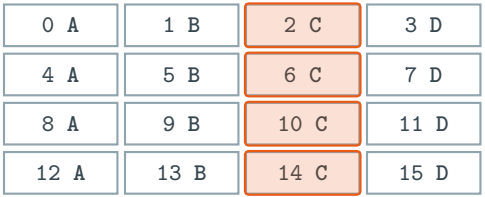
A column round




```
A: quarter_round(0, 4, 8, 12)
B: quarter_round(1, 5, 9, 13)
C: quarter_round(2, 6, 10, 14)
D: quarter_round(3, 7, 11, 15)
```

Column round: four quarter rounds, each on one column of the state.

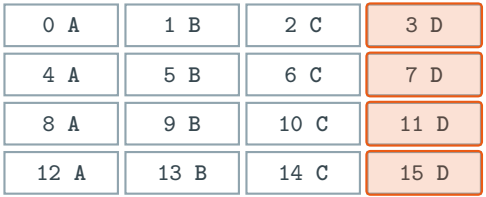
A column round




```
A: quarter_round(0, 4, 8, 12)
B: quarter_round(1, 5, 9, 13)
C: quarter_round(2, 6, 10, 14)
D: quarter_round(3, 7, 11, 15)
```

Column round: four quarter rounds, each on one column of the state.

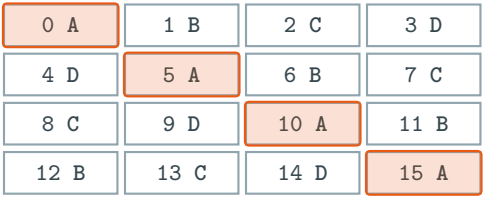
A column round




```
A: quarter_round(0, 4, 8, 12)
B: quarter_round(1, 5, 9, 13)
C: quarter_round(2, 6, 10, 14)
D: quarter_round(3, 7, 11, 15)
```

Column round: four quarter rounds, each on one column of the state.

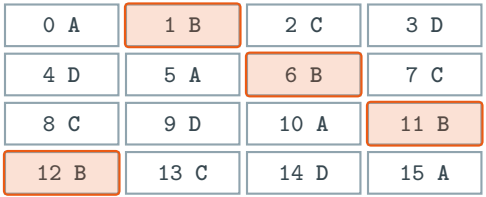
A diagonal round




```
A: quarter_round(0, 5, 10, 15)
B: quarter_round(1, 6, 11, 12)
C: quarter_round(2, 7, 8, 13)
D: quarter_round(3, 4, 9, 14)
```

Diagonal round: four quarter rounds, each on one diagonal; a diagonal wraps around the edge.

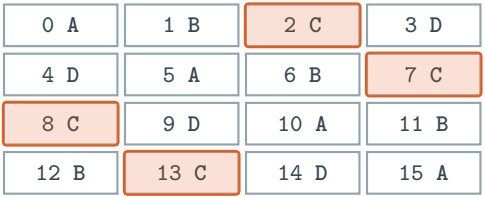
A diagonal round




```
A: quarter_round(0, 5, 10, 15)
B: quarter_round(1, 6, 11, 12)
C: quarter_round(2, 7, 8, 13)
D: quarter_round(3, 4, 9, 14)
```

Diagonal round: four quarter rounds, each on one diagonal; a diagonal wraps around the edge.

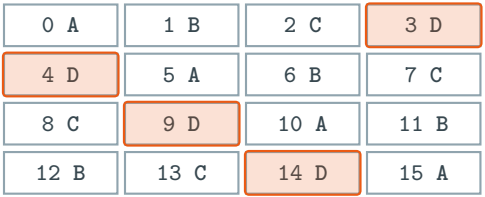
A diagonal round




```
A: quarter_round(0, 5, 10, 15)
B: quarter_round(1, 6, 11, 12)
C: quarter_round(2, 7, 8, 13)
D: quarter_round(3, 4, 9, 14)
```

Diagonal round: four quarter rounds, each on one diagonal; a diagonal wraps around the edge.

A diagonal round




```
A: quarter_round(0, 5, 10, 15)
B: quarter_round(1, 6, 11, 12)
C: quarter_round(2, 7, 8, 13)
D: quarter_round(3, 4, 9, 14)
```

Diagonal round: four quarter rounds, each on one diagonal; a diagonal wraps around the edge.

Two rounds: every word meets every other

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

After a column round
word 0 depends on words 0, 4, 8, 12

After a diagonal round
word 0 depends on **all 16 words**

A column round and a diagonal round together are a **double round**.

One double round connects every word with every other word.

Twenty rounds

```
for _ in range(10):  # 10 double rounds
    column round
    diagonal round
```

The RFC example after 20 rounds:

837778ab	e238d763	a67ae21e	5950bb2f
c4f2d0c7	fc62bb2f	8fa018fc	3f5ec7b7
335271c2	f29489f3	eabda8fc	82e46ebd
d19c12b4	b04e16de	9e83d0cb	4e3c50a2

Nothing looks like the constants or the key any more.

Add the initial state

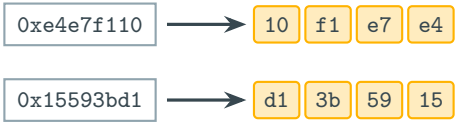
Each word: the word after 20 rounds + the same word of the initial state (mod 2^{32}).

```
word 0:    0x837778ab   +   0x61707865   =   0xe4e7f110
word 1:    0xe238d763   +   0x3320646e   =   0x15593bd1
...
word 15:   0x4e3c50a2   +   0x00000000   =   0x4e3c50a2
```

Why this addition matters, we will see by attacking it.

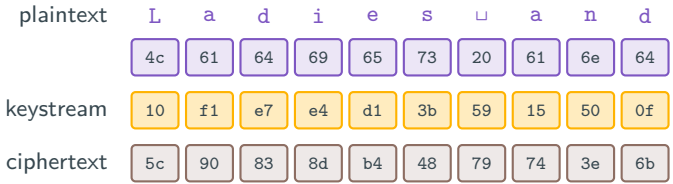
From words to bytes

Each of the 16 words is written as 4 bytes, little-endian:



The block: `10 f1 e7 e4 d1 3b 59 15 50 0f dd 1f a3 20 71 c4 ...` (64 bytes)

XOR with the plaintext



The block function makes the keystream. Encryption is still one XOR.

Task 6: one full block

Task 6
Write the whole block function of ChaCha20.

1. the initial state
2. 10 double rounds
3. add the initial state
4. 64 bytes, little-endian

Test: the example from RFC 8439, the block must start with `10 f1 e7 e4`.

Task 6: what to do

```python
def chacha20_block(key, counter, nonce):
    # TODO 1: the initial state, 16 words:
    #         CONSTANTS + 8 key words + counter + 3 nonce words
    #         struct.unpack("<8I", key) reads 8 little-endian words
    start = []

    # TODO 2: copy the state and run 10 double rounds on the copy:
    #   a column round:
    #     (0, 4, 8, 12) (1, 5, 9, 13) (2, 6, 10, 14) (3, 7, 11, 15)
    #   a diagonal round:
    #     (0, 5, 10, 15) (1, 6, 11, 12) (2, 7, 8, 13) (3, 4, 9, 14)
    s = start[:]

    # TODO 3: add the initial state to the result, word by word, with add32

    # TODO 4: return the 16 words as 64 bytes: struct.pack("<16I", *s)
    return b""
```

Task 6

Open `block.py` from **GitHub**.

Ready in the file

`quarter_round(s, a, b, c, d)`: it changes the list `s` in place.

Solution: one full block

```
start = CONSTANTS + list(struct.unpack("<8I", key)) \
        + [counter] + list(struct.unpack("<3I", nonce))
s = start[:]
for _ in range(10):
    quarter_round(s, 0, 4, 8, 12)    # column round
    ...                               # 3 more columns
    quarter_round(s, 0, 5, 10, 15)   # diagonal round
    ...                               # 3 more diagonals
s = [add32(x, y) for x, y in zip(s, start)]
return struct.pack("<16I", *s)
```

1 to 4
the four parts from the task, in order.

Shortened on the slide
the solution file has all 8 calls of `quarter_round`.

Solution: the output

```
got:        10f1e7e4d13b5915500fdd1fa32071c4c7d1f4c733c068030422aa9ac3d46c4e...
expected:   10f1e7e4d13b5915500fdd1fa32071c4c7d1f4c733c068030422aa9ac3d46c4e...
            ✔ OK
```

Your ChaCha20 gives the same block as the standard.

All of ChaCha20 on one slide

```python
def quarter_round(s, a, b, c, d):
    s[a] = add32(s[a], s[b]); s[d] ^= s[a]; s[d] = rotl32(s[d], 16)
    s[c] = add32(s[c], s[d]); s[b] ^= s[c]; s[b] = rotl32(s[b], 12)
    s[a] = add32(s[a], s[b]); s[d] ^= s[a]; s[d] = rotl32(s[d], 8)
    s[c] = add32(s[c], s[d]); s[b] ^= s[c]; s[b] = rotl32(s[b], 7)


def initial_state(key, counter, nonce):
    return ([0x61707865, 0x3320646E, 0x79622D32, 0x6B206574]
            + list(struct.unpack("<8I", key))
            + [counter]
            + list(struct.unpack("<3I", nonce)))
```

```python
def block(key, counter, nonce, rounds=20):
    start = initial_state(key, counter, nonce)
    s = start[:]
    run_rounds(s, rounds)
    s = [add32(x, y) for x, y in zip(s, start)]
    return struct.pack("<16I", *s)


def chacha20(key, nonce, counter, data):
    out = bytearray()
    for i in range(0, len(data), 64):
        ks = block(key, counter + i // 64, nonce)
        out += bytes(x ^ y for x, y in zip(data[i:i + 64], ks))
    return bytes(out)
```

Not shown: `add32`, `rotl32` and `run_rounds` (the column and diagonal rounds).

This is `chacha20_ready.py` from Task 1.

How many rounds are enough?



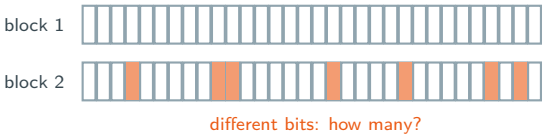
Why 20 rounds, and not 2?

Why add the initial state at the end?

Let us **measure** first, then **attack**.

Task 7: watch one bit spread

Task 7
Two blocks with the same key and nonce. The counters differ in **one bit**. How many of the 512 output bits are different?



after 1 round: **?**
after 2 rounds: **?**
after 20 rounds: **?**

Predict first, on paper. Then measure.

Task 7: what to do

```python
for rounds in [1, 2, 3, 4, 5, 6, 8, 12, 20]:
    total = 0
    for _ in range(TRIALS):
        key = random.randbytes(32)
        nonce = random.randbytes(12)
        counter = random.getrandbits(32)
        out1 = block(key, counter, nonce, rounds)
        out2 = block(key, counter ^ 1, nonce, rounds)
        # TODO: count how many of the 512 bits differ
        #       between out1 and out2, add it to total
    print(f"{rounds:2} rounds: {total / TRIALS:6.1f} of 512 bits changed")
```

Task 7

Open `avalanche.py` from **GitHub**.

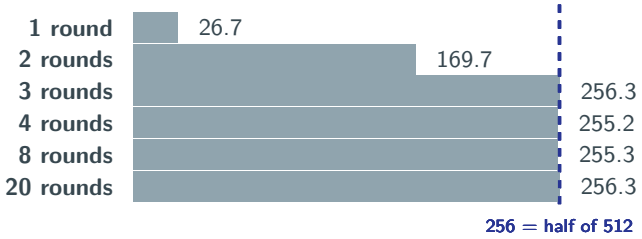
Hints

`int.from_bytes(out1, "big")` turns 64 bytes into one number.

`bin(x).count("1")` counts the 1 bits of `x`.

Solution: watch one bit spread

```
diff = int.from_bytes(out1, "big") ^ int.from_bytes(out2, "big")
total += bin(diff).count("1")
```



average over 200 random keys, of 512 bits

Why exactly half?

Two random blocks of 512 bits:
each bit is the same with chance $1/2$, different with chance $1/2$
so on average $512/2 = \mathbf{256}$ bits differ.



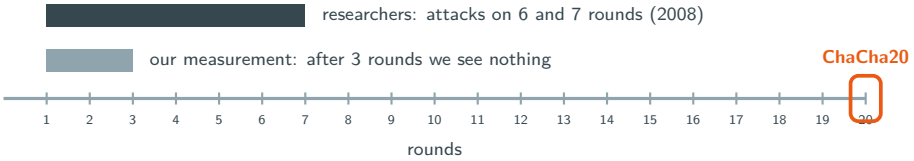
a coin for every bit

Avalanche: one changed input bit changes about half of the output bits.

After 3 rounds, one bit of difference looks like two unrelated blocks.

One changed input bit changes about half of the output bits.

Looks random is not the same as secure

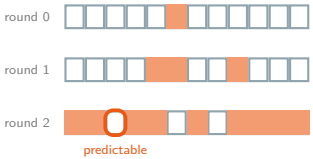


A simple count is a lower limit. Clever attacks see much further.

Passing one test does not prove a cipher secure.

Differential cryptanalysis: the idea

1. Choose two inputs with a chosen difference, for example one bit of the counter.
2. Follow the difference through the rounds.
3. Find an output bit that changes more often or less often than half the time.



Differential cryptanalysis: if such a bit exists, the output is not random, and the attacker learns about the key.

Linear cryptanalysis: the idea

Linear cryptanalysis: look for a XOR of a few input bits and output bits that is 0 more often than half the time.

XOR and rotation
keep such relations exact

Addition
breaks them, a little in every round

After enough rounds, no useful relation is left.

The best known attacks

rounds	best known attack	brute force
6	about 2^{139} operations (2008)	2^{256}
7	about 2^{248} operations (2008)	2^{256}
8	no attack known	2^{256}
20	no attack known	2^{256}

Brute force: try every possible key, 2^{256} of them.

Even 7 rounds are far from broken in practice. 20 rounds is a safety margin.

ChaCha8, ChaCha12, ChaCha20

variant	rounds	rounds above the best attack	1 MiB in plain Python
ChaCha8	8	1	1.00 s
ChaCha12	12	5	1.44 s
ChaCha20	20	13	2.24 s

Fewer rounds: faster. More rounds: a bigger safety margin.

Fewer rounds in real systems



ChaCha20 the standard choice on the internet



ChaCha12 Adiantum: disk encryption on cheap Android phones (2019)



ChaCha8 proposed in “Too Much Crypto” (Aumasson, 2019), not used by the big standards

Rounds can be undone

Every step of a round has an inverse:

```
a = a + b        ⟷   a = a - b
d = d ^ a        ⟷   d = d ^ a
d = rotl(d, 16)  ⟷   d = rotr(d, 16)
```

Whoever knows the state after 20 rounds can go back to the start.

The rounds hide nothing on their own.

Task 8: why add the initial state?

Task 8
A **broken** ChaCha20: 20 rounds, but no addition at the end. You intercepted one output block.

ed7caf84	ec59ab67	182353f4	51b53075
afb4b0a5	9d9a8a93	dd172de6	23a7a16e
38e2e4ca	a52d3212	2d79c87f	382257ec
83607414	11654b3c	a14846fd	36134694

Find the secret key.

Task 8: what to do

```python
def inverse_quarter_round(s, a, b, c, d):
    # TODO: undo the quarter round. Go through its 12 steps backwards:
    #   a += b; d ^= a; d = rotl(d, 16)
    #   c += d; b ^= c; b = rotl(b, 12)
    #   a += b; d ^= a; d = rotl(d, 8)
    #   c += d; b ^= c; b = rotl(b, 7)
    # The last step goes first: b = rotr32(b, 7), then b ^= s[c],
    # then c = sub32(c, d), ...
    # Use s[a], s[b], s[c], s[d] (the list s is changed in place).
    pass


s = output[:]
for _ in range(10):
    # undo one diagonal round, then one column round
    inverse_quarter_round(s, 3, 4, 9, 14)
    ...                                  # 7 more calls, ready
```

Task 8

Open `invert.py` from **GitHub**.

Ready in the file

`sub32` and `rotr32`, and the loop that undoes the rounds.

The program prints rows 1 to 3 of the state.

Solution: the inverse quarter round

```
def inverse_quarter_round(s, a, b, c, d):
    s[b] = rotr32(s[b], 7);  s[b] ^= s[c]; s[c] = sub32(s[c], s[d])
    s[d] = rotr32(s[d], 8);  s[d] ^= s[a]; s[a] = sub32(s[a], s[b])
    s[b] = rotr32(s[b], 12); s[b] ^= s[c]; s[c] = sub32(s[c], s[d])
    s[d] = rotr32(s[d], 16); s[d] ^= s[a]; s[a] = sub32(s[a], s[b])
```

the original, read from the bottom:

```
1  a += b;  d ^= a;  d = rotl(d, 16)
2  c += d;  b ^= c;  b = rotl(b, 12)
3  a += b;  d ^= a;  d = rotl(d, 8)
4  c += d;  b ^= c;  b = rotl(b, 7)
```

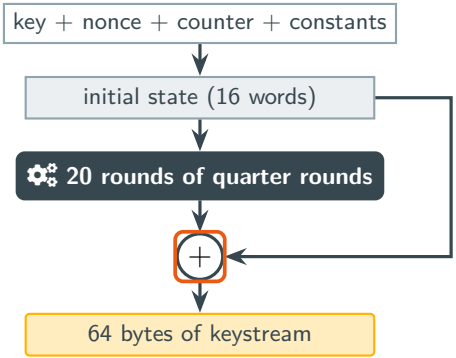


Solution: the key is back

constants:	61707865 3320646e 79622d32 6b206574	"expand 32-byte k"
key:	b'correct horse battery staple!!!!'	

Without the final addition, one block gives away the key.

Why the addition stops the attack



$$\text{output} = \text{rounds}(\text{start}) + \text{start}$$

To go back, the attacker must remove `start` first.

But `start` contains the key: the attacker does not know it.

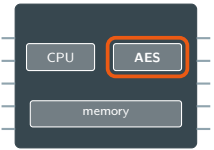
The rounds can be undone. Undoing rounds plus addition needs the key.

A **one-way** function: easy to compute forward, impossible in practice to undo.

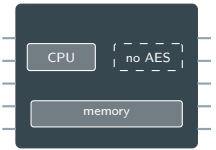
Adding the initial state turns a reversible mixer into a one-way function.

Many processors have no AES instructions

AES: the most used **block cipher** today (it encrypts blocks of 16 bytes).



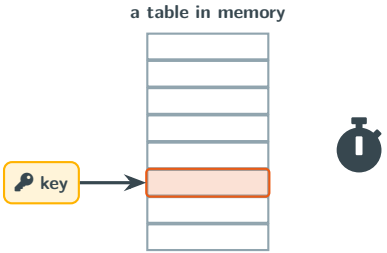
laptops, new phones



cheap phones, watches, routers

Laptops and new phones have special AES instructions in the processor.
Cheap phones, watches, routers and small devices often do not.

AES in software: tables and timing



Fast AES without special instructions uses **tables in memory**.

Which table entry is read depends on the key.

Reading memory takes more or less time, depending on the cache.

An attacker who measures the time can learn about the key.

Timing attack: the attacker learns a secret by measuring how long the computation takes.

Fast software AES can leak the key through timing.

ChaCha20 needs nothing special



only add, rotate, XOR — every processor has them



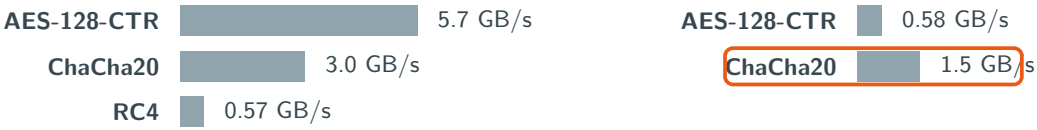
no tables in memory — the time does not depend on the key



the same work on many words — fits the vector instructions of phones

ChaCha20 is fast and safe in plain software.

Measured on a laptop



OpenSSL 3.0.13, Intel Core i5-8265U, AES instructions switched off by `OPENSSL_ia32cap`

Without AES instructions, ChaCha20 is about 2.6 times faster than AES.

Faster than RC4, without its weaknesses

	RC4	**ChaCha20**
speed (laptop)	0.57 GB/s	3.0 GB/s
output bias	✗ yes, measurable	✓ none known
nonce	✗ none, glued on by hand	✓ built in
random access	✗ no	✓ yes, by counter

ChaCha20 does what RC4 did, faster and safely.

Why Google switched phones to ChaCha20



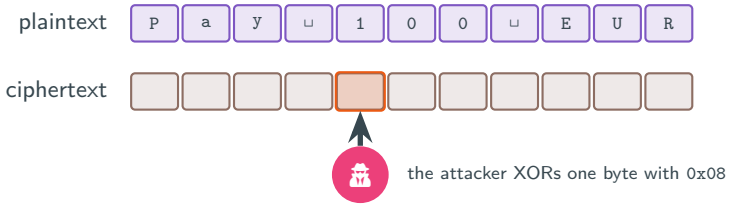
2014: Chrome on Android starts to use ChaCha20 for **HTTPS** to Google.

Reason given by Google:

about 3 times faster than AES on devices without AES instructions, and RC4 was already broken.

Without AES instructions, ChaCha20 is the fast choice.

Recall: encryption does not protect integrity



Integrity: the certainty that nobody changed the message.

Encryption hides the message, but does not stop changes.

A tag

The sender computes a short **tag** from the ciphertext and a secret key.



Poly1305 is the function that computes such a tag.

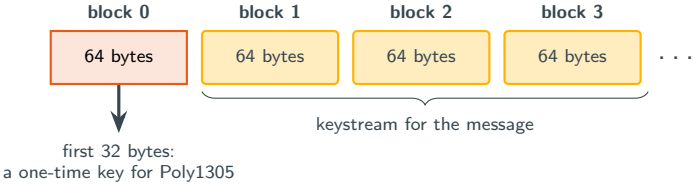
A changed byte gives a different tag.

Authenticated encryption

Authenticated encryption (**AEAD**): encryption and a tag in one operation.
Decryption first checks the tag, and refuses to decrypt if it is wrong.



Where the Poly1305 key comes from



A new nonce gives a new Poly1305 key. One key per message.

This is why ChaCha20-Poly1305 starts encrypting at counter 1.

Task 9: change one byte

Task 9

Message: `Pay 100 EUR to Anna.` Encrypted twice: only ChaCha20, and ChaCha20-Poly1305.



only ChaCha20 | ChaCha20-Poly1305

Change "1" into "9" in both ciphertexts (XOR byte 4 with 0x08)
and try to decrypt them.

Task 9: what to do

```
# Without a tag: only ChaCha20, the data starts at counter 1
plain = chacha20(key, nonce, 1, message)
# With a tag: ChaCha20-Poly1305
sealed = seal(key, nonce, message)

# The attacker changes "1" (0x31) into "9" (0x39): XOR 0x08 at position 4.
# TODO 1: make changed_plain: a copy of plain with byte 4 XORed with 0x08
#         hint: b = bytearray(plain); b[4] ^= 0x08; changed_plain = bytes(b)
# TODO 2: make changed_sealed in the same way from sealed
# TODO 3: decrypt changed_plain with chacha20(key, nonce, 1, ...) and print it
# TODO 4: decrypt changed_sealed with unseal(key, nonce, ...) inside
#         try: ... except TagError as e: print("rejected:", e)
```

Shortened on the slide: the import, the key, the nonce and one print are in the file.

Task 9

Open `tamper.py` from **GitHub**.

Ready

`aead.py`: `chacha20`, `seal`, `unseal`, `TagError`

Solution: change one byte

without a tag: ✗ `Pay 900 EUR to Anna.`

with a tag: ✓ `rejected: the tag does not match: the message was changed`

With Poly1305, the change is caught. Without it, the change goes through.

The nonce rule still holds

The same key and nonce used twice with ChaCha20-Poly1305:

⚠ **the same keystream**
→ P1 XOR P2 leaks

⚠ **the same Poly1305 key**
→ the attacker can make valid tags

A tag does not fix a repeated nonce. It makes the damage worse.

One key and nonce pair, one message. Always.

Good to know: XChaCha20



96 bits: choosing it at random is risky after many messages



192 bits

XChaCha20: a nonce of 192 bits, so a random nonce is safe.
Used for example in libsodium and in WireGuard's cookies.

Where you use it: the web



TLS 1.3 (2018) `TLS_CHACHA20_POLY1305_SHA256`, one of three that all should support



Chrome and Android since 2014, on devices without AES instructions

TLS: the protocol that encrypts HTTPS.

Where you use it: VPN and SSH



WireGuard a VPN, uses only ChaCha20-Poly1305 for its data



OpenSSH 6.5 (2014) `chacha20-poly1305@openssh.com`, today the default

VPN: an encrypted tunnel between a computer and a network. **SSH**: an encrypted login to a remote computer.

Where you use it: phones and the kernel



Adiantum (2019) disk encryption on cheap Android phones, with ChaCha12



Linux 4.8 (2016) `/dev/urandom` produces random numbers with ChaCha20

RC4 and ChaCha20 side by side

	RC4	ChaCha20-Poly1305
nonce	✗ none	✓ 96 bits, built in
random access	✗ no	✓ yes, by counter
known bias	✗ yes	✓ none for 20 rounds
integrity	✗ none	✓ Poly1305 tag
status	✗ forbidden in TLS	✓ in TLS 1.3, SSH, WireGuard

Every weakness of RC4 is fixed in ChaCha20-Poly1305.

Summary

1. Key, nonce and counter give any 64-byte block of the keystream directly.
2. Inside are only three operations on 32-bit words: add, rotate, XOR.
3. ChaCha20: a 4 by 4 state, 20 rounds of quarter rounds, add the initial state.
4. Rounds spread every bit; the final addition stops the rounds being undone.
5. Fast without special hardware, and with Poly1305 it also protects integrity.